Trio Motion Technology

Robert Lucas - 2025/26 Placement
BSc(Hons) Computer Science with Professional Placement

Trio: Industrial Automation

Trio Controller

The Brains



Trio Controller

- Closed-loop motion control up to 64 axes
- Inverse kinematics, path interpolation, etc.
- User-programmed logic
- Multitasking/processing, bare-metal RTOS
- Networking protocols
- IDE integration

Other Outputs



Controller with I/O expansions

125 µs

Closed-Loop Cycle Time



Trio Servo Drives

Servo Drives

'Dumb' Motor Control / Power

Other Inputs



Controller with I/O expansions



Servo Motor

Protocols

Modbus

EtherCAT®

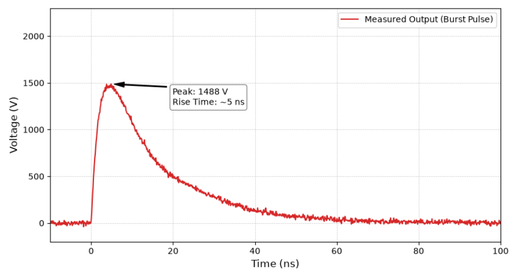
CANopen®

EtherNet/IP™

Not Ethernet or IP!

Hardware

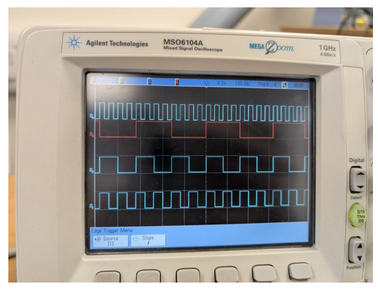
(2 months, 2nd rotation)



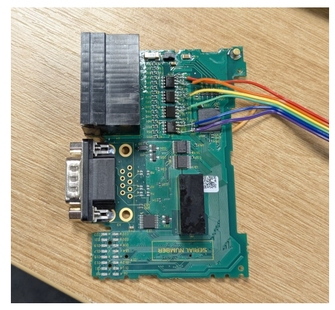
Typical EFT generator pulse

Spent some time diagnosing product EFT issues

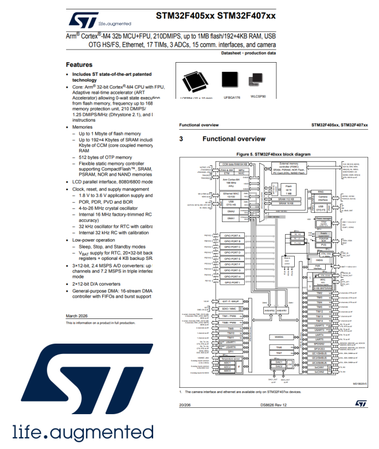
CPU Selection — Most of my time in Hardware was spent selecting a CPU for a new product, including measuring the performance of representative C workloads on a test rig. This taught me a lot about various CPU technologies and their trade-offs.



CPU response time testing



Creating a test rig



Comparing CPU Hardware for Realtime Requirements

Applications

(3 months, 3rd rotation)

SPS Demo — I spent the majority of my time in Applications working on a demo machine using Trio products for the SPS trade show in Italy. In addition to gaining experience in creating these kinds of machines, it also gave me a different view of our products from the end user's perspective, improving my work when I returned to Software.



Finished Rig at SPS 2026 Italy



Wall of drives at Trio UK for high axis-count testing

Coding in an environment where mistakes could cause physical damage to the machine and delay the project took a lot of adjusting to, coming from an environment where testing is risk-free, allowing fast iteration.

Testing — For my last few weeks in Applications (continuing into Software), I worked on a Python framework for performing integration/regression tests on hardware. Python was a great choice here for its fast iteration, platform independence, and broad ecosystem which provided support for a lot of protocols we wanted to test. However, its performance shortfalls required careful design of test code, and may in the future require us to reach for C extensions.

Software

(7 months, 1st and 4th rotation)

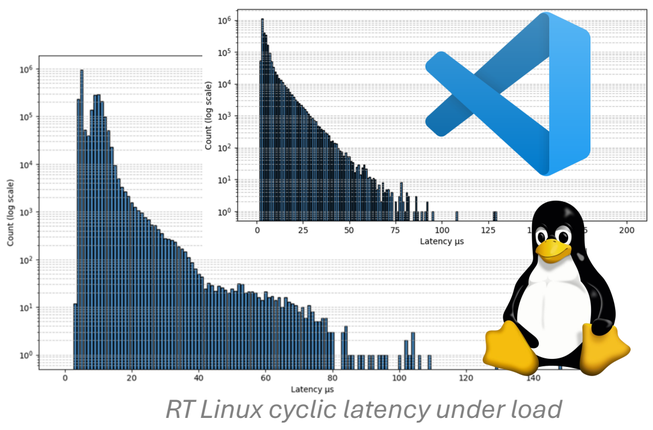
Bare Metal OS — Working on a bare metal OS with indefinite uptime and strict performance requirements presented several unique challenges not present in other development, such as there being no OS 'safety net', minimising allocation, and debugging on-hardware rather than on my local machine. In addition to this, the complexity of the project required a lot of communication with coworkers, a skill I believe I greatly improved this year.



Trio's 'Motion Perfect' IDE

Bare Metal Python — In my time at Software, I completed a project investigating implementing high-level languages the end user could use to program the controller. The demo implementation of my choice, Python (via MicroPython), required a lot of work to port to our OS and to extend it with debugging features MicroPython does not support.

RT Linux — Another project I worked on was investigating the performance of RTOSs, how they could be applied to our use-case, and, particularly for RT Linux, its cyclic latency performance under load. I found this project especially interesting as RT Linux appears to be gaining traction, with PREEMPT_RT patches only being mainlined in 2024.



RT Linux cyclic latency under load

EtherCAT Work — My final months at Trio were spent working on our implementation of EtherCAT—a protocol enabling real-time cyclic data transfer over standard Ethernet cables. Work on both this and Python required tight iteration with our IDE team, with considerations of ease-of-implementation (and usually, therefore, maintenance) vs. the state management and parsing burden on the IDE.